



Trabalho Prático

Computação Gráfica - Fase III

2025/2026

Grupo 50

Francisco Contente - A106846 Gustavo Braga - A107379 Lucas Pinto - A107288

Índice Geral

1. Introdução	1
2. Engine	2
2.1 Utilização de VBOs	2
2.1.1 Leitura e Carregamento de Dados	2
2.1.2 Inicialização dos VBOs	3
2.1.3 Renderização Hierárquica	3
2.2 Curvas de Catmull-Rom	3
2.3 Transformações: Translação, Rotação e Align	4
3. Generator	6
3.1 Patches de Bézier	6
4. Cometa e Sistema Solar	8
4.1. Criação do Cometa com Patches de Bézier	8
4.2. Sistema Solar Dinâmico	9
5. Resultado dos Testes	12
6. Conclusão	13

Índice de Figuras

Figura 1 - Cometa (Perspetiva 1)	9
Figura 2 - Cometa (Perspetiva 2)	9
Figura 3 - Sistema Solar Dinâmico	11
Figura 4 - Teste 1	12
Figura 5 - Teste 2	12

1. Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Computação Gráfica (CG), do 3.º ano, 2.º semestre, do curso de Licenciatura em Engenharia Informática (LEI), no ano letivo 2025/2026, e descreve o trabalho desenvolvido na Fase 3 do projeto.

Esta fase consistiu na atualização do *Generator* para suportar superfícies de Bézier e na otimização do *Engine* através da utilização de VBOs (*Vertex Buffer Objects*). Adicionalmente, foram implementadas translações e rotações animadas, sendo as translações baseadas em curvas de Catmull-Rom com a funcionalidade *Align*, que permite que os objetos mantenham a sua orientação alinhada com a direção do movimento.

2. Engine

2.1 Utilização de VBOs

Nesta fase do projeto, a arquitetura do motor de renderização (Engine) sofreu uma atualização significativa com a implementação de **Vertex Buffer Objects (VBOs)**. O objetivo principal desta transição foi otimizar o desempenho gráfico da aplicação, abandonando o modo imediato (*glBegin / glEnd*) em favor de uma abordagem mais eficiente.

Para acomodar esta nova lógica de renderização, a estrutura de dados Model, responsável por armazenar a informação individual de cada modelo 3D na memória, foi redesenhada:

```
struct Model {  
    std::string filePath;  
    GLuint vertices[1] = {0};  
    unsigned int vertexCount = 0;  
};
```

Esta estrutura passou a incluir o caminho para o ficheiro que contém a geometria (*filePath*), um identificador associado ao buffer de vértices gerado pelo OpenGL (*vertices*) e ainda o número total de vértices (*vertexCount*).

Para complementar a estrutura Model e gerir de forma eficiente a fase de carregamento dos dados, foi introduzida uma estrutura auxiliar designada por ModelData:

```
struct ModelData {  
    std::vector<float> vertices;  
};
```

A função desta estrutura é estritamente temporária e atua como um intermediário durante o processo de leitura e preparação dos dados. O *std::vector<float> vertices* é utilizado para armazenar de forma contígua, na memória todas as coordenadas (X, Y e Z) lidos sequencialmente a partir do ficheiro .3d.

2.1.1 Leitura e Carregamento de Dados

O processo de leitura dos ficheiros é assegurado pela função *loadModelData*, que preenche a estrutura *ModelData*. Esta função começa por ler o número total de vértices presente no ficheiro e utiliza essa informação para pré-alocar memória no vetor, através do método *reserve()*. Esta abordagem evita múltiplas realocações durante a leitura, contribuindo para uma maior eficiência. De seguida, os valores das coordenadas são lidos sequencialmente e armazenados em memória.

2.1.2 Inicialização dos VBOs

A função *initGroupBuffers* é responsável por preparar os dados para utilização dos VBOs. Para cada modelo, os vértices são lidos do ficheiro e armazenados temporariamente em memória. De seguida, é criado um buffer na GPU através de *glGenBuffers*, sendo depois feita a cópia dos dados para a placa gráfica com *glBufferData*.

Este processo é executado apenas uma vez no início da execução, permitindo que os dados fiquem armazenados na GPU e não tenham de ser reenviados a cada frame.

2.1.3 Renderização Hierárquica

Com a introdução dos VBOs, a principal alteração na função *renderGroup* ocorreu na forma como os modelos são desenhados.

Na versão anterior, os vértices eram enviados um a um para o OpenGL dentro de um bloco *glBegin/glEnd*, utilizando *glVertex3f*. Este método é menos eficiente, pois implica múltiplas chamadas à API gráfica.

Na nova versão, os vértices deixam de ser enviados diretamente a cada frame. Em vez disso, o buffer previamente criado é ativado com *glBindBuffer*, e a forma como os dados estão organizados é definida com *glVertexPointer*. Por fim, o desenho é realizado com uma única chamada à função *glDrawArrays*, que processa todos os vértices de uma só vez.

2.2 Curvas de Catmull-Rom

As curvas de Catmull-Rom foram utilizadas para implementar movimentos suaves ao longo de trajetórias definidas por pontos de controlo. Este tipo de curva tem a particularidade de interpolar diretamente os pontos fornecidos, permitindo que o objeto passe exatamente por cada um deles.

A posição de um ponto na curva é calculada a partir de quatro pontos de controlo consecutivos e de um parâmetro $t \in [0, 1]$, através da seguinte expressão:

$$p(t) = [t^3 \ t^2 \ t^1 \ 1] \cdot M \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

onde M representa a matriz característica das curvas de Catmull-Rom:

$$M = \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1.0 & -2.5 & 2.0 & -0.5 \\ -0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \end{bmatrix}$$

Esta abordagem encontra-se implementada na função *getCatmullRomPoint*, onde os pontos de controlo são separados por coordenadas (X, Y e Z) e processados individualmente. A multiplicação da matriz pelos pontos é realizada através da função auxiliar *multMatrixVector*, obtendo-se assim os coeficientes da equação cúbica. A posição final é depois calculada substituindo o valor de t , sendo também calculada a derivada da função, que corresponde à tangente da curva nesse ponto.

Para permitir a utilização de vários pontos de controlo ao longo de toda a trajetória, foi implementada a função *getGlobalCatmullRomPoint*. Esta função converte um parâmetro global $gt \in [0, 1]$ num segmento específico da curva, determinando quais os quatro pontos relevantes para o cálculo. A utilização do operador módulo garante que os índices são sempre válidos, permitindo ainda que a curva seja fechada e contínua.

2.3 Transformações: Translação, Rotação e Align

Após a introdução das curvas de Catmull-Rom, foi necessário adaptar o sistema de transformações de forma a suportar movimentos dinâmicos ao longo dessas trajetórias.

No caso das translações, estas podem assumir duas formas distintas. Quando não são definidos pontos de controlo, a transformação é estática, sendo aplicada diretamente através de *glTranslatef* com os valores fornecidos. Nas translações com animação, quando são definidos pontos de controlo e um tempo associado, a posição do objeto deixa de ser fixa e passa a ser calculada ao longo da curva em função do tempo decorrido. Esse tempo é obtido através de *glutGet(GLUT_ELAPSED_TIME)* e convertido para segundos. De forma a garantir que a animação é cíclica e normalizada entre 0 e 1, é aplicada a seguinte expressão:

$$t = \frac{t_{elapsed} \bmod T}{T}$$

onde $t_{elapsed}$ corresponde ao tempo total decorrido desde o início da execução e T ao tempo definido para a transformação. Este parâmetro t é então utilizado na função *getGlobalCatmullRomPoint*, que calcula a posição correspondente na curva de Catmull-Rom. O resultado é aplicado através de *glTranslatef*, permitindo que o objeto percorra continuamente a trajetória definida pelos pontos de controlo.

Relativamente às rotações, estas também podem ser estáticas ou dinâmicas. Numa rotação estática, é aplicado um ângulo fixo em torno de um eixo, utilizando *glRotatef*. Já no caso dinâmico, quando é definido um tempo, o ângulo de rotação é calculado em função do tempo decorrido, sendo calculado da seguinte forma:

$$\theta(t) = t_{elapsed} \bmod T \cdot \frac{360}{T}$$

permitindo uma rotação contínua e uniforme ao longo do tempo.

Adicionalmente, foi implementado o mecanismo de *align*, que permite alinhar automaticamente o objeto com a direção da curva. Para isso, é utilizada a derivada calculada nas curvas de Catmull-Rom, que define o vetor tangente à trajetória. Este vetor é normalizado e usado como eixo principal de orientação. A partir dele, e com recurso a produtos vetoriais, são calculados vetores ortogonais que permitem construir uma base de coordenadas local. Essa base é convertida numa matriz de rotação através da função *buildRotMatrix* e aplicada com *glMultMatrixf*.

3. Generator

3.1 Patches de Bézier

Nesta fase do projeto foi implementado suporte para superfícies de Bézier, permitindo a geração de geometria mais suave a partir de um conjunto de pontos de controlo.

Cada patch de Bézier corresponde a uma grelha de 16 pontos de controlo, organizados numa matriz 4x4. A avaliação destes pontos baseia-se nos polinómios de *Bernstein*, que permitem calcular a contribuição de cada ponto de controlo em função dos parâmetros u e v , ambos no intervalo $[0,1]$. Os polinómios de *Bernstein* de grau 3 são definidos como:

$$B_0(t) = (1 - t)^3$$

$$B_1(t) = 3t(1 - t)^2$$

$$B_2(t) = 3t^2(1 - t)$$

$$B_3(t) = t^3$$

Com base nestes polinómios, um ponto na superfície de Bézier bicúbica é calculado através da soma ponderada dos 16 pontos de controlo:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u)B_j(v)P(i, j)$$

Na implementação desenvolvida, este cálculo encontra-se na função *evaluateBezierPatch*, que aplica os polinómios de *Bernstein* aos parâmetros u e v , e combina os pontos de controlo correspondentes, resultando na posição final da superfície.

Os dados de entrada são lidos pela função *readBezierPatches*, que interpreta o ficheiro de patches. Este ficheiro contém, por um lado, os índices dos pontos de controlo associados a cada patch e, por outro, a lista global de pontos no espaço 3D. Os índices são depois utilizados para construir a matriz 4x4 de cada patch, permitindo associar corretamente cada superfície aos seus pontos.

Após a leitura, a geração da geometria é feita na função *generateBezier*. Cada patch é subdividido numa grelha de pontos definida pelo nível de tesselação. Através da variação sucessiva dos valores u e v no domínio $[0,1] \times [0,1]$, são calculados os pontos correspondentes no espaço tridimensional, formando uma grelha uniforme de pontos sobre a superfície.

De seguida, cada célula da grelha é dividida em dois triângulos, garantindo a representação da superfície através de primitivas compatíveis com o OpenGL. Todos os vértices gerados são armazenados num vetor e posteriormente escritos para um ficheiro de saída através da função *writeVerticesToFile*.

4. Cometa e Sistema Solar

4.1. Criação do Cometa com Patches de Bézier

Para a construção do cometa foi desenvolvido no *Generator* o **comet_patch.cpp**, responsável por criar um ficheiro de patches de Bézier, que contém a descrição geométrica do objeto sob a forma de pontos de controlo e da sua organização em patches bicúbicos.

O processo inicia-se com a definição de uma parametrização da superfície através de coordenadas $(u, v) \in [0, 1]^2$, que representam posições sobre uma esfera (análogas a latitude e longitude). Cada par (u, v) é convertido num ponto tridimensional pertencente a uma esfera unitária através de coordenadas esféricas:

$$x = \sin(\theta)\cos(\phi) \quad y = \cos(\phi) \quad z = \sin(\theta)\sin(\phi)$$

onde $\theta = u\pi$ e $\phi = 2\pi v$. Esta formulação permite obter uma superfície contínua e uniformemente parametrizada.

De seguida, a geometria base é deformada de forma a simular um corpo irregular. Esta deformação é definida em função dos parâmetros (u, v) , através de um fator radial $r(u, v)$, que controla a distância de cada ponto ao centro da esfera. A nova posição é então dada por:

$$p'(u, v) = r(u, v) p(u, v)$$

O fator radial resulta da combinação de vários componentes:

- **Formação de ruído**, definido como uma soma de funções trigonométricas de diferentes frequências (função *rockyNoise*).
- **Crateras**, modeladas como funções radiais centradas em pontos (c_u, c_v) . Para cada ponto da superfície, calcula-se a distância normalizada ao centro da cratera:

$$d = \sqrt{(u - c_u)^2 + (v - c_v)^2}$$

O processo de definição da cratera combina termos exponenciais e polinomiais, cada um responsável por diferentes zonas da cratera, desde o núcleo até às bordas, criando assim regiões côncavas e convexas no cometa.

- **Transformações globais**, como escalas anisotrópicas nos eixos (x, y e z), que deformam a geometria base de uma esfera para uma forma elipsoidal, e pequenas perturbações adicionais que introduzem assimetria e evitam uma aparência regular.

O valor final de $r(u, v)$ é limitado a um intervalo definido, garantindo que a geometria não sofre inversões ou degenerações.

Após a deformação, os pontos são organizados numa grelha bidimensional regular cuja dimensão segue o padrão $3n + 1$, permitindo a sua divisão em patches de Bézier bicúbicos 4×4 . Cada patch é definido por 16 pontos de controlo e partilha fronteiras com os patches adjacentes, garantindo continuidade geométrica da superfície.

O ficheiro `.patch` resultante contém o número total de patches, a lista de índices (16 por patch) e a lista completa dos pontos de controlo.

Este é posteriormente processado pelo módulo **bezier.cpp**, para gerar o modelo final em formato `.3d`, utilizado na renderização.

Para gerar e renderizar o cometa, foram executados os seguintes comandos, na pasta **build/Release**:

```
.\generator comet patches/comet.patch  
.\generator bezier patches/comet.patch 15 models/comet.3d  
.\engine scenes/comet.xml
```

Obtém-se assim o modelo final do cometa:

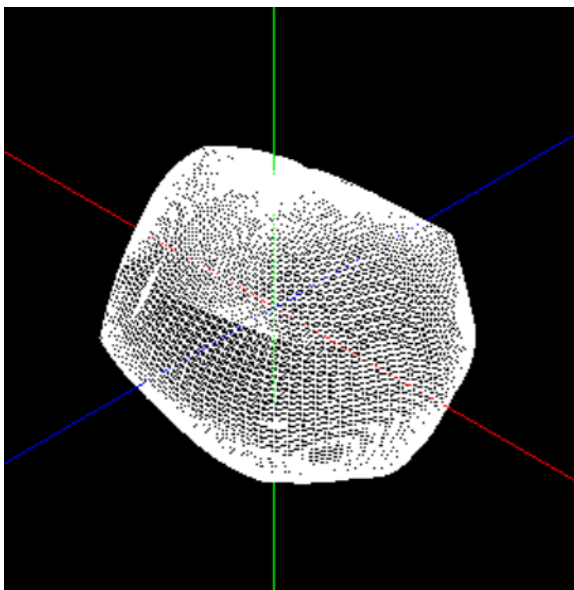


Figura 1 - Cometa (Perspetiva 1)

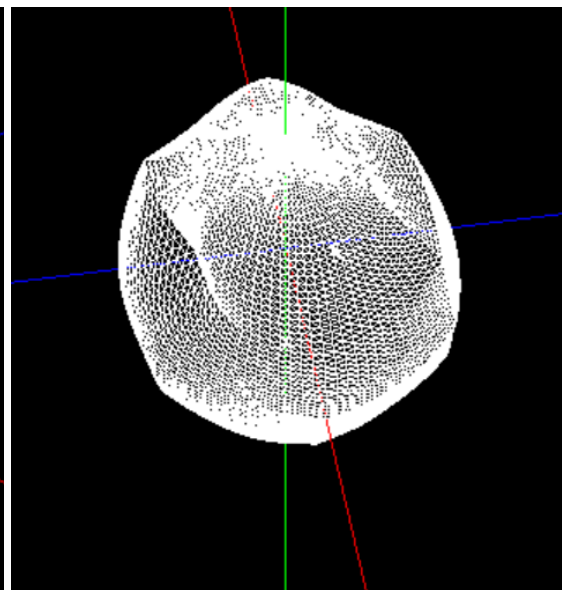


Figura 2 - Cometa (Perspetiva 2)

4.2. Sistema Solar Dinâmico

Nesta fase, a construção do sistema solar diferiu significativamente da abordagem utilizada na fase anterior. Enquanto que na Fase 2 a cena foi definida de forma manual e com um

pequeno programa para gerar o cinturão de asteróides, nesta fase foi desenvolvido, dentro do Generator, o ficheiro **solar_generator.cpp**, responsável por produzir automaticamente o ficheiro XML completo da cena.

Com esta abordagem, passou a ser possível definir de forma estruturada todos os elementos do sistema solar (planetas, satélites, cinturão de asteroides e cometa) bem como as respetivas transformações, garantindo consistência e facilidade na configuração dos vários parâmetros.

Relativamente às animações, foram utilizadas trajetórias baseadas em curvas de Catmull-Rom para descrever o movimento dos planetas em torno do Sol. Para tal, foram gerados vários pontos igualmente espaçados ao longo de uma circunferência, segundo a parametrização:

$$x = r \cos(\theta) \quad z = r \sin(\theta) \quad \theta = \frac{2\pi i}{N}$$

Esses pontos definem a trajetória seguida por cada planeta ao longo do tempo, sendo interpretados pelo elemento `<translate time="...">` como uma curva fechada e contínua.

No caso do cometa, foi utilizada também uma trajetória baseada em Catmull-Rom, mas definida manualmente através de um conjunto de pontos não coplanares, permitindo criar um percurso irregular. Adicionalmente, foi utilizado o atributo `align="True"`, garantindo que o modelo se orienta automaticamente segundo a direção da curva ao longo do movimento.

Para além do movimento de translação, foi também considerado o movimento de rotação dos planetas sobre si próprios. Em particular, foram introduzidas variações na orientação do eixo de rotação em casos específicos, como em Vénus (rotação no sentido inverso) e Urano (eixo significativamente inclinado), com o objetivo de trazer ao cenário um aspeto mais realista.

Por outro lado, em situações mais simples, como na movimentação da Lua e do cinturão de asteróides, não foram utilizadas trajetórias baseadas em curvas de Catmull-Rom. Nestes casos, o movimento de translação resulta da aplicação de rotações ao longo do tempo, combinadas com a estrutura hierárquica da cena.

No caso da Lua, a sua órbita em torno da Terra é obtida através da aplicação de um *rotate time* seguida de uma translação que define a distância ao planeta, fazendo com que esta descreva uma trajetória circular sobre a Terra.

Já no cinturão de asteróides, os objetos encontram-se previamente distribuídos no espaço, sendo aplicado apenas um *rotate time* ao grupo, o que provoca um movimento orbital em torno do centro do sistema. O cinturão foi ainda reutilizado da Fase 2, sendo apenas ajustada a sua escala para se integrar no novo sistema solar.

As primitivas utilizadas neste cenário foram obtidas através dos seguintes comandos:

```

.\generator sphere 1 20 20 sphere.3d
.\generator torus 1.2 0.2 20 20 torus.3d
.\generator box 2 10 box.3d
.\generator cone 1 2 10 10 cone.3d
.\generator comet patches/comet.patch
.\generator bezier patches/comet.patch 15 comet.3d

```

Para gerar o XML final do sistema solar e visualizá-lo, devem ser executados os seguintes comandos, na pasta **build/Release**:

```

.\generator solar solar_fase3.xml ../../scenes/belt.xml
.\engine scenes/solar_fase3.xml

```

Obtém-se assim a cena resultante do sistema solar dinâmico, com movimentos contínuos e hierarquicamente organizados. Foi ainda incluído um vídeo demonstrativo do resultado final que poderá ser encontrado em **videos/solar_fase3.mp4**.

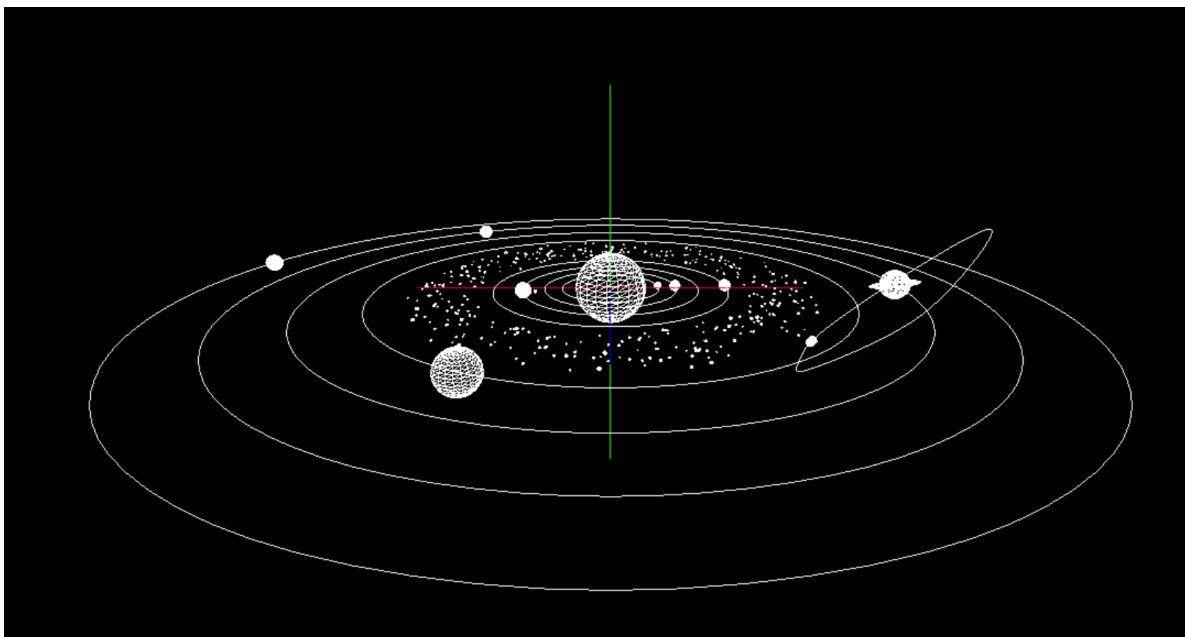


Figura 3 - Sistema Solar Dinâmico

5. Resultado dos Testes

Nesta secção apresentam-se os resultados obtidos na execução dos testes fornecidos para a Fase 3. Estes testes permitem validar o correto funcionamento das novas funcionalidades introduzidas, nomeadamente o suporte a curvas de Catmull-Rom para animações, rotações dependentes do tempo e geração de superfícies através de patches de Bézier. Os resultados demonstram que a engine interpreta corretamente os novos elementos do ficheiro XML, aplicando animações ao longo de trajetórias definidas por curvas e rotações contínuas com base no tempo especificado.

Para o teste 1, para além da imagem apresentada abaixo, foi incluído um vídeo demonstrativo do resultado obtido, que poderá ser encontrado em ***videos/teste_3_1.mp4***.

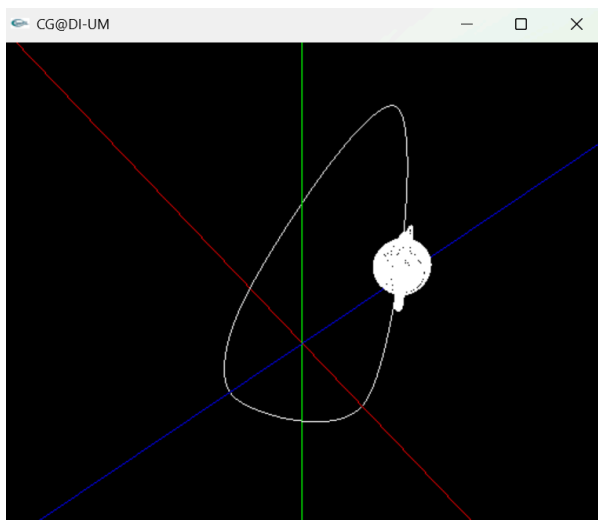


Figura 4 - Teste 1

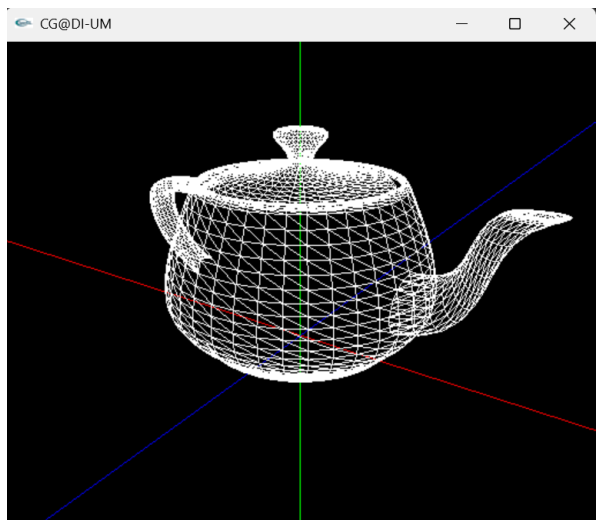


Figura 5 - Teste 2

6. Conclusão

Na terceira fase do projeto de Computação Gráfica foram desenvolvidas funcionalidades mais avançadas ao nível da modelação e animação de cenas tridimensionais. Os principais objetivos desta fase, nomeadamente o suporte a curvas de Catmull-Rom para animações, a geração de superfícies através de patches de Bézier e a utilização de VBOs para renderização, foram implementados com sucesso.

A integração de trajetórias baseadas no tempo permitiu criar movimentos contínuos, enquanto que a utilização de superfícies de Bézier possibilitou a construção de modelos mais complexos, como o cometa. Por fim, a adoção de VBOs contribuiu para a melhoria do desempenho do motor gráfico na renderização dos modelos.

Estas funcionalidades foram validadas através da construção de uma cena dinâmica do sistema solar e do desenvolvimento do modelo do cometa com recurso a patches de Bézier, onde foi possível aplicar de forma integrada todos os conceitos abordados. Considera-se, assim, que os objetivos definidos para esta fase foram atingidos, e que o trabalho desenvolvido se encontra preparado para as futuras modificações a ocorrer na última fase do projeto.